TactfulLLM: Learning Context-Aware Proactivity Calibration

Anonymous Author(s)
Affiliation
Address
email

Abstract

Nowadays, people increasingly view Large Language Models (LLMs) as collaborative partners, expecting them to proactively ask users for clarification when task specifications are incomplete or ambiguous, with varying expectations about how proactive the model should be, rather than remain passive tools.

However, current models typically ask excessive questions or execute without clarification, relying on fixed proactivity strategies that fail to accommodate the diverse collaboration preferences of users.

To address these limitations, we introduce TactfulLLM, a novel and lightweight training framework that enhances active LLM-human collaboration by achieving appropriate proactivity.

The core idea formalizes proactive engagement not as a fixed intensity, but as a dynamic, high-level "Clarify-or-Execute" decision in each dialogue turn.

This decision is optimized through a preference signal that balances final task success against the cumulative cost of interruption, allowing the model to solve tasks while respecting different users' preferred levels of assistance.

Experiments on conversational code generation tasks demonstrate that TactfulLLM significantly adapts to user personas.

Our policy outperforms fixed baselines, achieving X% improvement in task success and a Y% reduction in user rejection rate, validating that effective adaptive proactivity can be learned efficiently.

1 Introduction

Large Language Models (LLMs) are increasingly used as true collaborative partners Zou et al. [2025b,a], Deng et al. [2025], across writing Mysore et al. [2025], coding Murali et al. [2024], and problem solving Huang et al. [2025]. Diverging from their previous role as passive tools that merely follow user-issued instructions, modern LLM workflows often necessitate bi-directional interactions Shen et al. [2024, 2025], in which models proactively identify missing information, ask clarifying questions, propose next steps, and help users refine their intents. Effective collaboration, however, depends not only on a model's capability to generate solutions, but also on how proactive the model should be when collaborating with users Chen et al. [2025].

Despite growing expectations for collaborative behavior, current LLMs exhibit static and poorly calibrated proactive patterns. Some models ask too many clarifying questions, interrupting users and slowing progress Zhang et al. [2024]; others execute prematurely without verification, risking incorrect assumptions or irrelevant outputs Li et al. [2023]. These fixed strategies fail to accommodate the wide diversity in how users prefer to collaborate: some users value exhaustive guidance, while others prioritize rapid iteration Desolda et al. [2025]. While recent work has improved proactive engagement in LLM interactions via intent modeling, user simulations, or multi-turn reward optimization Lu

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

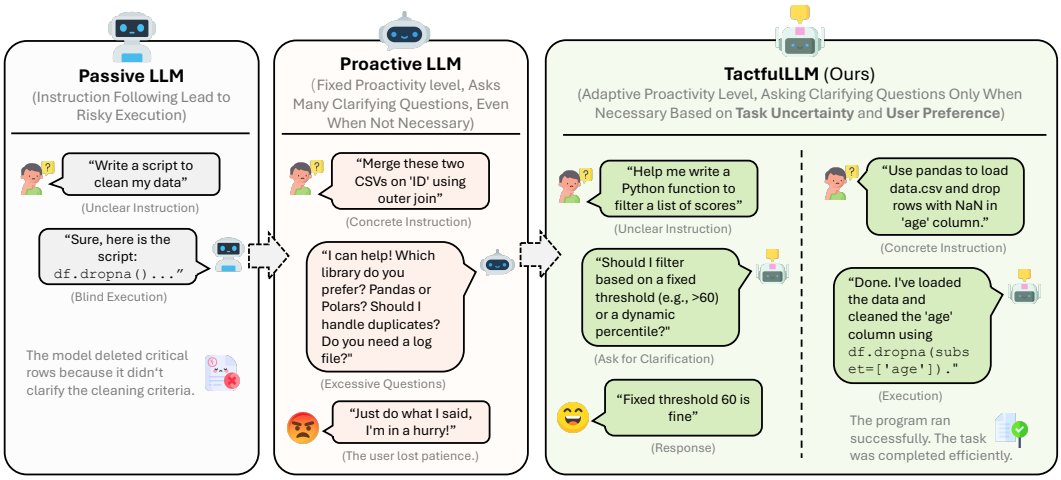

Figure 1: Comparison of LLM interaction styles. Traditional instruction-following LLMs tend to execute immediately under unclear requests (left), while recent proactive LLMs adopt a fixed level of clarification that may over-interrupt some users (middle). In contrast, **TactfulLLM** adaptively adjusts its proactivity based on the conversation context and user preferences, asking clarifying questions only when appropriate (right).

et al. [2024], Wu et al. [2025], Qian et al. [2025], these methods focus on training models toward generally proactive behaviors and do not explicitly address how to adaptively regulate proactivity during interaction based on user preferences and task state.

In this work, we introduce TactfulLLMs (see figure 1), a lightweight training framework that enables LLMs to adapt their level of proactivity to both task context and individual user preferences

2 Related Work

2.1 Proactive Decision-making in LLMs

Existing research on LLM proactivity follows two primary paradigms: inference-time prompting and training-based optimization. While prompting methods rely on structured instructions to steer model behavior, training-based approaches modify model parameters to internalize proactive decision-making across interactions.

Inference-time Prompting for Proactivity. Early work primarily explores inference-time prompting strategies that steer models toward proactive clarification or planning behaviors through structured in-context instructions.

For example, Proactive Chain-of-Thought embeds clarification decisions directly into the model's reasoning process by prompting the model to explicitly assess task ambiguity and decide whether additional user input is required before acting Deng et al. [2023]. Related work on clarification question generation further models task ambiguity explicitly by maintaining multiple candidate task interpretations and selecting clarification questions that are expected to maximally reduce uncertainty among them Kobalczyk et al. [2025]. While effective, inference-time prompting encodes proactive behavior as fixed reasoning patterns and remains highly sensitive to prompt design, offering limited ability to adapt clarification decisions to interaction context or user preferences.

Training-based Proactive Decision-Making. To overcome the limitations of prompting, recent studies internalize proactivity through Supervised Fine-Tuning (SFT) and Reinforcement Learning (RL). While SFT grounds behavior on annotated interactions Li et al. [2025], it often yields rigid policies that fail to weigh the long-term utility of resolving ambiguity against the immediate cost of user interruption

Subsequent research leverages RL to optimize proactive decision-making for long-horizon success. Specifically, CollabLLM focuses on optimizing overall collaborative effectiveness by simulating

multi-turn interactions, where proactive behaviors like clarification emerge implicitly as a byproduct of improving long-term collaboration quality Wu et al. [2025]. Building on this goal of maximizing task success, Zhang et al. further refine the decision-making process by explicitly modeling clarification as a strategic single-turn choice; they compare potential future conversation trajectories under "asking" versus "answering" to determine which path yields higher downstream success Zhang et al. [2025]. Taking a broader systemic view, UserRL formulates human–agent interaction as a Markov Decision Process (MDP) and trains policies with per-turn feedback from simulated users to maximize cumulative task success over extended dialogues, without isolating clarification as a distinct decision Qian et al. [2025].

Despite these advances, prior training-based approaches leave the decision of whether to ask for clarification implicit, treating it as a byproduct of task optimization. However, prior surveys emphasize that the timing and necessity of initiation, that is, when a system chooses to be proactive, plays a critical role in shaping user trust, often outweighing the content of the initiation itself Deng et al. [2025]. Without explicit modeling of when to initiate clarification, helpful clarification can easily become intrusive when it misaligns with the user's goals or tolerance. In contrast, we model proactivity as an explicit Clarify-or-Execute meta-decision at each dialogue turn, directly optimizing the trade-off between uncertainty reduction and user interruption.

2.2 Human-AI Collaboration and Preference Alignment

From Outcome Alignment to Process-Based Alignment. Most existing preference alignment frameworks encode human preferences at the level of task outcomes Jiang et al. [2025], such as correctness, helpfulness Ouyang et al. [2022], or aggregate reward Christiano et al. [2017]. In these formulations, alignment is predominantly evaluated at the level of final outputs or accumulated reward, whereas the interaction process where the model decides when and how to intervene is not explicitly modeled and is treated as secondary Rafailov et al. [2023]. Recent work on collaborative systems suggests that user satisfaction and perceived alignment are shaped not only by task outcomes, but also by how interaction unfolds over time Siu et al. [2021], such as communication patterns and behavioral consistency. This line of work highlights a complementary perspective on alignment that emphasizes the role of interaction processes in collaborative settings Mayer et al. [2025], McDonald and Gonzalez [2025], Siu et al. [2025].

How User Personas Shape Collaboration Preferences. A complementary line of research highlights that collaboration preferences are heterogeneous across users Carroll et al. [2019]. Differences in expertise, experience, and situational context lead users to evaluate agent behavior differently, even when task outcomes are similar Pitis et al. [2024], Liang et al. [2024]. For example, expert users may prefer minimal guidance and greater autonomy, whereas novices often benefit from more structured support. These findings challenge one-size-fits-all alignment strategies and motivate adaptive approaches that infer and respond to latent user-specific preferences Chen et al. [2024], Zollo et al. [2024]. Despite growing interest in personalization, reliably capturing and representing such behavioral preferences remains an open challenge in alignment research Xie et al. [2025].

3 Problem Formulation

In real-world human–AI collaboration scenarios, user requests are often underspecified. For example, in conversational collaborative coding, important details such as input constraints and output formats may not be fully expressed in the initial instruction. As a result, the assistant must decide whether to proceed with execution using the available information or to ask clarification questions to obtain additional requirements.

We model this interaction as a multi-turn conversational process. A user has an underlying task goal g, which is only partially specified in the initial request. The interaction unfolds over multiple turns $t_j = \{u_j, m_j\}$, where u_j denotes the user input and m_j denotes the model response at turn j, for $j = 1, \ldots, K$. At each turn, the assistant observes the current interaction context and must decide between two possible actions, taking into account user preferences for interaction:

- **Clarify**: ask a clarification question to obtain additional information about the task.
- **Execute**: directly generate a solution based on the currently available specification.

The conversation terminates when the assistant executes the task or when the user chooses to end the interaction. The objective of the assistant is to generate a sequence of responses $\{m_j\}_{j=1}^{K}$ that successfully completes the user's task while maintaining efficient interaction.

4 Method

We model clarification in human–AI collaboration as a sequential decision-making problem. Given an underspecified task request, the assistant must decide whether to ask clarification questions to obtain missing information or directly execute the task based on the currently available specification.

Our framework learns this decision policy from simulated interaction trajectories. Each trajectory captures a multi-turn interaction between an assistant and a simulated user, including clarification attempts, user responses, and the final task outcome.

Based on these signals, we construct trajectory-level preferences and train a clarification policy using supervised imitation and preference optimization.

The overall training pipeline consists of three components: (1) a decision policy that determines when to clarify or execute, (2) a reward formulation that balances task success and interruption cost,and (3) a trajectory simulation framework that generates interaction data for policy learning.

4.1 Decision Policy

We model clarification as a sequential decision process. At interaction step j, the assistant observes a dialogue state s_j, which summarizes the current task specification and interaction context.

Formally, the state is defined as

$$s_j = (q_j, \sigma_j, d_j, r_j, p),$$

where q_j denotes the current task description that accumulates conversation history, $\sigma_j \in [0, 1]$ represents the task uncertainty score reflecting how incomplete the specification is, d_j is the current dialogue turn number, $r_j \in \{0, 1\}$ indicates whether the previous action was rejected by the user, and p encodes the user persona.

Given the state, the assistant must choose between two possible actions:

- **Clarify**: ask a clarification question to obtain additional information about the task.
- **Execute**: generate a task solution using the currently available specification.

We parameterize this decision process using a policy π_θ that maps dialogue states to an action distribution:

$$a_j \sim \pi_\theta(a_j \mid s_j), \quad a_j \in \{\text{Clarify}, \text{Execute}\}.$$

Intuitively, the policy learns when clarification is beneficial for reducing task uncertainty and when direct execution is preferable to avoid unnecessary interaction overhead.

4.2 Reward Design

The goal of the assistant is to successfully complete the user's task while minimizing unnecessary interruptions during interaction. We therefore define a trajectory-level reward that balances task success and interaction efficiency.

The reward for an interaction trajectory τ is defined as

$$R(\tau) = w_{task} R_{task} - w_{interrupt} C_{interrupt},$$

where R_{task} measures the success of the final task execution and $C_{interrupt}$ captures the cumulative cost introduced by clarification.

Task Success. For coding tasks, R_{task} is measured by the functional correctness of the generated solution. If the solution passes all associated test cases, we set $R_{\text{task}} = 1$; otherwise, the reward is proportional to the fraction of tests passed. Crucially, R_{task} is a terminal reward computed only at the final execution step and is propagated back to all preceding turns in the trajectory.

Interruption Cost. Clarification may introduce interaction overhead and disrupt users who prefer efficient communication. We therefore define an interruption cost that accumulates across dialogue turns:

$$C_{interrupt} = \sum_{j=1}^{K} c(a_j, r_j).$$

The per-turn cost function is defined as

$$c(a_j, r_j) = \lambda - \gamma\alpha_j + \delta r_j,$$

where $\alpha_j \in \{0, 1\}$ indicates whether the user answered the clarification question and $r_j \in \{0, 1\}$ indicates whether the user rejected it.

Answering a clarification question reduces the cost (encouraging informative clarification), while rejection increases the cost (penalizing unnecessary interruptions).

Multi-turn Credit Assignment. For multi-turn trajectories, all turns share the final task outcome R_{task} computed from the final Execute action, while interruption costs accumulate across turns. This design enables early clarification actions to receive credit if they ultimately contribute to successful task completion, while still penalizing excessive or ineffective clarification.

4.3 Trajectory Generation with Simulated Users

To train the clarification policy, we construct a simulation framework that generates multi-turn interactions between the assistant and synthetic users. The framework converts standard task benchmarks into simulated collaborative interactions between LLM and simulated users.

Each interaction trajectory consists of a sequence of assistant actions, user responses, and the final task outcome.

The simulation pipeline consists of three components:

1. Task masking to create underspecified task requests.
2. User personas that model heterogeneous collaboration preferences.
3. A user simulator that generates responses to clarification actions.

Task Masking and State Construction To simulate realistic collaborative settings where task descriptions are incomplete, we start with fully specified tasks and mask critical information to produce underspecified task requests. Specifically, we mask information such as input constraints, output format requirements, edge cases, and validation rules from the original task description.

For each masked task, we construct an initial dialogue state s_1. This state includes the masked task description q_1, an initial task uncertainty score $\sigma_1 \in [0, 1]$ computed heuristically from the masked query text (based on factors such as query length, the presence of question marks, and ambiguous keywords), the dialogue turn index $d_1 = 0$, and a reject indicator $r_1 = 0$ indicating that no clarification rejection has occurred yet. The query representation q_1 initially contains only the masked task description, and conversation history is progressively accumulated into q_j as the interaction proceeds. The user persona p is sampled according to the persona configuration.

The masked information is stored in a disclosure rule that is accessible only to the user simulator. When the assistant asks clarification questions, the simulator may reveal parts of this hidden information according to the persona's expertise level.

4.3.1 User Personas

To model heterogeneous user preferences in collaborative interactions, we characterize users using two interpretable parameters: *patience* and *expertise*.

To model heterogeneous user preferences in collaborative interactions, we characterize users using two interpretable parameters: *patience* and *expertise*. Patience determines the probability that a user

answers a clarification question, and this probability gradually decreases as the dialogue progresses. Expertise captures the user's ability to provide precise and informative responses; users with higher expertise reveal a larger portion of the missing task requirements in each clarification round.

These two parameters provide a compact abstraction of user behavior and allow the simulator to generate diverse interaction styles. Different user personas can then be instantiated as specific combinations of these parameters.

4.3.2 User Simulator

The user simulator models the behavioral responses of different personas to the assistant's actions during interaction. When the assistant issues a clarification request, the simulator determines whether the user answers or rejects the request.

The probability that the user answers a clarification request is modeled as

$$P(\text{answer} \mid p, t) = p_{\text{base}} \cdot \gamma^{t},$$

where p_{base} represents the persona's base patience level, t is the dialogue turn index, and $\gamma < 1$ models exponential patience decay across dialogue turns. The rejection probability is defined as $P(\text{reject}) = 1 - P(\text{answer})$.

Persona-specific behavioral adjustments are applied to reflect different interaction styles (e.g., lower tolerance for clarification or rejection of redundant questions). Detailed simulator configurations are provided in Appendix A.3.

4.4 Preference Construction and Policy Optimization

We construct preference comparisons from simulated interaction trajectories generated for the same task instance. Each trajectory is evaluated using the trajectory-level reward defined in Section 4.2, which balances task success and interaction cost.

For supervised fine-tuning (SFT), we use the generated trajectories as imitation data. The assistant actions observed in the trajectories are treated as supervision targets, allowing the policy to learn clarification behaviors through behavioral imitation.

For preference optimization, we construct pairwise comparisons between trajectories. Given two trajectories τ^{+} and τ^{-}, the trajectory with higher reward is treated as the preferred one. We then apply Direct Preference Optimization (DPO) to train the policy to assign higher probability to actions appearing in preferred trajectories compared to rejected ones.

This training process encourages the policy to learn clarification decisions that improve task success while avoiding unnecessary interaction interruptions.

5 Experiment

5.1 Experimental Setup

Task and Data. We evaluate TactfulLLM on conversational code generation tasks using BigCodeBench Zhuo et al. [2024]. From the original 1,140 tasks, we select 470 tasks that exhibit inherent ambiguity (e.g., missing edge case specifications in instructions). We construct unspecified inputs by masking critical information, including input constraints, output formats, and edge cases. The masked descriptions serve as initial user queries, while the hidden information is retained in disclosure rules accessible only to the simulated user during interaction. Details of the filtering and masking procedures are provided in Appendix A.1.

Execution Settings. During trajectory generation, when the model selects the Execute action, we generate three versions of the solution to enable comparative analysis: (1) **Direct Execution**, generated from the initial masked query without any clarification; (2) **Clarified Execution**, generated from the masked query augmented with information obtained when the user answers clarification questions; and (3) **Oracle Execution**, generated from the original, full task description in the benchmark. These settings allow us to quantify how much clarification recovers the performance gap caused by underspecified inputs.

User Personas. We instantiate three representative personas with different combinations of patience and expertise: *Busy Developer* (moderate expertise, low patience), *Novice Learner* (low expertise, high patience), and *Experienced Engineer* (high expertise, moderate patience). These personas enable evaluation of whether the model adapts its behavior to different user characteristics. Additional details are provided in Appendix A.2.

Baselines. We compare TactfulLLM with several baselines representing different proactivity strategies. **Direct Execution** never asks for clarification, while **Always Clarify** asks clarification questions at every step. **Heuristic Policy** follows a rule-based strategy based on task uncertainty and dialogue context. **SFT Policy** is trained via supervised fine-tuning on simulated trajectories without preference optimization. TactfulLLM is trained using Direct Preference Optimization (DPO) over trajectory-level comparisons.

Evaluation Metrics. We evaluate both task performance and interaction efficiency. **Task Success** measures whether the generated solution passes all test cases, and **Partial Pass Rate** measures the fraction of tests passed. To capture interaction cost, we report the average number of clarification questions, user rejection rate, and dialogue length. We also report the average trajectory reward defined in Section 4.2. Finally, we introduce **Oracle Gap Recovery (OGR)**, which measures how much clarification recovers the performance loss caused by missing information:

$$\text{OGR} = \frac{S_{\text{clarified}} - S_{\text{direct}}}{S_{\text{oracle}} - S_{\text{direct}} + \epsilon}.$$

5.2 Results

Our evaluation focuses on whether TactfulLLM improves the trade-off between task success and interaction cost, and whether it learns to apply clarification selectively based on task uncertainty.

Overall Performance

Effect of Clarification

Proactivity Calibration

5.3 Ablation Study

Acknowledgments

Use unnumbered first level headings for the acknowledgments. All acknowledgments, including those to funding agencies, go at the end of the paper.

Ethics Statement

Authors can add an optional ethics statement to the paper. For papers that touch on ethical issues, this section will be evaluated as part of the review process. The ethics statement should come at the end of the paper. It does not count toward the page limit, but should not be more than 1 page.

References

Micah Carroll, Rohin Shah, Mark K Ho, Tom Griffiths, Sanjit Seshia, Pieter Abbeel, and Anca Dragan. On the utility of learning about humans for human-ai coordination. *Advances in neural information processing systems*, 32, 2019.

Ruizhe Chen, Xiaotian Zhang, Meng Luo, Wenhao Chai, and Zuozhu Liu. Pad: Personalized alignment at decoding-time. In *The Thirteenth International Conference on Learning Representations*, 2024.

Valerie Chen, Alan Zhu, Sebastian Zhao, Hussein Mozannar, David Sontag, and Ameet Talwalkar. Need help? designing proactive ai assistants for programming. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, pages 1–18, 2025.

Paul F Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/d5e2c0adad503c91f91df240d0cd4e49-Paper.pdf.

Yang Deng, Lizi Liao, Liang Chen, Hongru Wang, Wenqiang Lei, and Tat-Seng Chua. Prompting and evaluating large language models for proactive dialogues: Clarification, target-guided, and non-collaboration. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 10602–10621, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.711. URL https://aclanthology.org/2023.findings-emnlp.711/.

Yang Deng, Lizi Liao, Wenqiang Lei, Grace Hui Yang, Wai Lam, and Tat-Seng Chua. Proactive conversational ai: A comprehensive survey of advancements and opportunities. *ACM Transactions on Information Systems*, 43(3):1–45, 2025.

Giuseppe Desolda, Andrea Esposito, Francesco Greco, Cesare Tucci, Paolo Buono, and Antonio Piccinno. Understanding user mental models in ai-driven code completion tools: Insights from an elicitation study. *International Journal of Human-Computer Studies*, 205:103648, 2025. ISSN 1071-5819. doi: https://doi.org/10.1016/j.ijhcs.2025.103648.

Tenghao Huang, Sihao Chen, Muhao Chen, Jonathan May, Longqi Yang, Mengting Wan, and Pei Zhou. Teaching language models to gather information proactively. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 15588–15599, 2025.

Ruili Jiang, Kehai Chen, Xuefeng Bai, Zhixuan He, Juntao Li, Muyun Yang, Tiejun Zhao, Liqiang Nie, and Min Zhang. A survey on human preference learning for aligning large language models. *ACM Comput. Surv.*, 58(6), December 2025. ISSN 0360-0300. doi: 10.1145/3773279. URL https://doi.org/10.1145/3773279.

Kasia Kobalczyk, Nicolás Astorga, Tennison Liu, and Mihaela van der Schaar. Active task disambiguation with LLMs. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=JAMxRSXLFz.

Haau-Sing Xiaocheng Li, Mohsen Mesgar, André FT Martins, and Iryna Gurevych. Python code generation by asking clarification questions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 14287–14306, 2023.

Xiaoyu Li, Xiao Li, Li Gao, Yiding Liu, Xiaoyang Wang, Shuaiqiang Wang, Junfeng Wang, and Dawei Yin. Proactive guidance of multi-turn conversation in industrial search. In Georg Rehm and Yunyao Li, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 6: Industry Track)*, pages 706–717, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-288-6. doi: 10.18653/v1/2025.acl-industry.50. URL https://aclanthology.org/2025.acl-industry.50/.

Yancheng Liang, Daphne Chen, Abhishek Gupta, Simon S Du, and Natasha Jaques. Learning to cooperate with humans using generative agents. *Advances in Neural Information Processing Systems*, 37:60061–60087, 2024.

Yaxi Lu, Shenzhi Yang, Cheng Qian, Guirong Chen, Qinyu Luo, Yesai Wu, Huadong Wang, Xin Cong, Zhong Zhang, Yankai Lin, et al. Proactive agent: Shifting llm agents from reactive responses to active assistance. *arXiv preprint arXiv:2410.12361*, 2024.

Lukas William Mayer, Sheer Karny, Jackie Ayoub, Miao Song, Danyang Tian, Ehsan Moradi-Pari, and Mark Steyvers. Human-ai collaboration: Trade-offs between performance and preferences. *arXiv preprint arXiv:2503.00248*, 2025.

Chase McDonald and Cleotilde Gonzalez. Controllable complementarity: Subjective preferences in human-ai collaboration. *arXiv preprint arXiv:2503.05455*, 2025.

Vijayaraghavan Murali, Chandra Maddila, Imad Ahmad, Michael Bolin, Daniel Cheng, Negar Ghorbani, Renuka Fernandez, Nachiappan Nagappan, and Peter C Rigby. Ai-assisted code authoring at scale: Fine-tuning, deploying, and mixed methods evaluation. *Proceedings of the ACM on Software Engineering*, 1(FSE):1066–1085, 2024.

Sheshera Mysore, Debarati Das, Hancheng Cao, and Bahareh Sarrafzadeh. Prototypical human-AI collaboration behaviors from LLM-assisted writing in the wild. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 16819–16846, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi: 10.18653/v1/2025.emnlp-main.852. URL https://aclanthology.org/2025.emnlp-main.852/.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.

Silviu Pitis, Ziang Xiao, Nicolas Le Roux, and Alessandro Sordoni. Improving context-aware preference modeling for language models. *Advances in Neural Information Processing Systems*, 37:70793–70827, 2024.

Cheng Qian, Zuxin Liu, Akshara Prabhakar, Jielin Qiu, Zhiwei Liu, Haolin Chen, Shirley Kokane, Heng Ji, Weiran Yao, Shelby Heinecke, Silvio Savarese, Caiming Xiong, and Huan Wang. Userrl: Training interactive user-centric agent via reinforcement learning, 2025.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems*, 36:53728–53741, 2023.

Hua Shen, Tiffany Knearem, Reshmi Ghosh, Kenan Alkiek, Kundan Krishna, Yachuan Liu, Savvas Petridis, Yi-Hao Peng, Li Qiwei, Chenglei Si, et al. Position: Towards bidirectional human-ai alignment. In *The Thirty-Ninth Annual Conference on Neural Information Processing Systems Position Paper Track*, 2024.

Hua Shen, Tiffany Knearem, Reshmi Ghosh, Michael Xieyang Liu, Andrés Monroy-Hernández, Tongshuang Wu, Diyi Yang, Yun Huang, Tanushree Mitra, Yang Li, et al. Bidirectional human-ai alignment: Emerging challenges and opportunities. In *Proceedings of the Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*, pages 1–6, 2025.

Ho Chit Siu, Jaime Peña, Edenna Chen, Yutai Zhou, Victor Lopez, Kyle Palko, Kimberlee Chang, and Ross Allen. Evaluation of human-ai teams for learned and rule-based agents in hanabi. *Advances in Neural Information Processing Systems*, 34:16183–16195, 2021.

Ho Chit Siu, Jaime D Pena, Yutai Zhou, and Ross E Allen. In pursuit of predictive models of human preferences toward ai teammates. *arXiv preprint arXiv:2503.15516*, 2025.

Shirley Wu, Michel Galley, Baolin Peng, Hao Cheng, Gavin Li, Yao Dou, Weixin Cai, James Zou, Jure Leskovec, and Jianfeng Gao. Collabllm: From passive responders to active collaborators. *arXiv preprint arXiv:2502.00640*, 2025.

Zhouhang Xie, Junda Wu, Yiran Shen, Yu Xia, Xintong Li, Aaron Chang, Ryan Rossi, Sachin Kumar, Bodhisattwa Prasad Majumder, Jingbo Shang, et al. A survey on personalized and pluralistic preference alignment in large language models. *arXiv preprint arXiv:2504.07070*, 2025.

Michael JQ Zhang, W Bradley Knox, and Eunsol Choi. Modeling future conversation turns to teach llms to ask clarifying questions. *arXiv preprint arXiv:2410.13788*, 2024.

Michael J.Q. Zhang, W. Bradley Knox, and Eunsol Choi. Modeling future conversation turns to teach llms to ask clarifying questions. *International Conference on Learning Representations (ICLR)*, 2025.

Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widyasari, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, et al. Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions. *arXiv preprint arXiv:2406.15877*, 2024.

Thomas P Zollo, Andrew Wei Tung Siah, Naimeng Ye, Ang Li, and Hongseok Namkoong. Personal-llm: Tailoring llms to individual preferences. *arXiv preprint arXiv:2409.20296*, 2024.

Henry Peng Zou, Wei-Chieh Huang, Yaozu Wu, Yankai Chen, Chunyu Miao, Hoang Nguyen, Yue Zhou, Weizhi Zhang, Liancheng Fang, Langzhou He, Yangning Li, Dongyuan Li, Renhe Jiang, Xue Liu, and Philip S. Yu. Llm-based human-agent collaboration and interaction systems: A survey, 2025a. URL `https://arxiv.org/abs/2505.00753`.

Henry Peng Zou, Wei-Chieh Huang, Yaozu Wu, Chunyu Miao, Dongyuan Li, Aiwei Liu, Yue Zhou, Yankai Chen, Weizhi Zhang, Yangning Li, Liancheng Fang, Renhe Jiang, and Philip S. Yu. A call for collaborative intelligence: Why human-agent systems should precede ai autonomy, 2025b. URL `https://arxiv.org/abs/2506.09420`.

A Appendix

A.1 Data Processing Details

This appendix provides detailed procedures for filtering and masking BigCodeBench tasks to create underspecified scenarios for training TactfulLLM.

A.1.1 Task Filtering

We filter tasks from BigCodeBench that exhibit inherent ambiguity, making them suitable for clarification scenarios. We use a rule-based filtering approach that identifies tasks where critical information is missing from the instruction but present in test cases.

A task is selected if it satisfies at least one of the following criteria:

1. Input constraints ambiguity: Test cases include edge cases (e.g., empty inputs, negative numbers, zero values) that are not explicitly mentioned in the instruction prompt.
2. Output format ambiguity: Test cases verify output types or formats (e.g., using `isinstance` checks) that are not specified in the instruction.
3. Multiple edge cases: Test cases contain three or more different edge case scenarios (empty, negative, zero, single element, null, etc.), indicating potential ambiguity.
4. Information gap: The instruction prompt is short (less than 200 characters) while the test suite is complex (more than 1000 characters), suggesting missing specifications.

From the original 1,140 tasks in BigCodeBench, this filtering process selects 470 tasks (41.2%) that exhibit inherent ambiguity.

A.1.2 Task Masking

For each selected task, we apply moderate-level masking to remove critical information from the instruction prompt. The masking process targets four categories of information:

1. Input constraints: Removes descriptions about handling edge cases (e.g., "handle empty inputs", "process negative numbers", "default is...") using regex pattern matching.
2. Output format: Removes detailed output specifications following "should output with:" statements, while preserving basic function signatures.
3. Edge cases: Extracts edge case information from test cases (e.g., empty input, negative numbers, zero values, single elements, duplicates) using keyword matching. All 470 tasks (100%) have edge cases extracted.
4. Validation rules: Extracts exception handling requirements (e.g., "raise ValueError if...") from instructions. 130 tasks (27.7%) contain validation rules.

The masking process uses regex patterns to identify and remove relevant text segments. The masked information is stored in a `disclosure_rule` structure that is accessible only to the user simulator during trajectory generation.

A.1.3 Masked Fields Statistics

Table 1 summarizes the distribution of masked information across the 470 tasks.

Field	Tasks with Content	Percentage
Edge cases	470	100.0%
Output format	456	97.0%
Validation rules	130	27.7%
Input constraints	60	12.8%

Table 1: Distribution of masked fields across 470 tasks.

The most common field combination is output format + edge cases (311 tasks, 66.2%), followed by output format + edge cases + validation rules (99 tasks, 21.1%). All tasks have at least two fields with content (edge cases and output format).

A.1.4 Disclosure Rule Structure

Each masked task includes a `disclosure_rule` that stores the masked information in two formats:

- `masked_fields`: Records the exact text segments that were removed from the original instruction, organized by category (input constraints, output format, edge cases, validation rules).
- `disclosure_info`: Provides structured information for the user simulator to use when answering clarification questions. This includes hints about edge cases, output specifications, and validation requirements.

The `disclosure_rule` is not visible to the policy model during training or inference. It is only used by the user simulator during trajectory generation to provide realistic responses to clarification questions based on the persona's expertise level.

A.2 Persona Configuration

A.3 User Simulator Details

A.4 Table of Notation

The following table summarizes the key mathematical symbols and variables used in the problem formulation and methodology of TactfulLLM.

Symbol	Definition and Description
Core MDP & Interaction	
K	Total number of turns in a single interaction trajectory.
j, t	Index of the current dialogue turn, where $j, t \in \{1, \ldots, K\}$ (j for state indexing, t for turn-based formulas).
s_j	The dialogue state at turn j, represented as a tuple $(q_j, \sigma_j, d_j, r_j, p)$.
a_j	The discrete action taken by the assistant, $a_j \in \{\text{Clarify}, \text{Execute}\}$.
m_t	Assistant message at turn t (clarification questions or solution).
o_t	User reaction at turn t (answer, reject, or silence).
π_θ	The decision policy parameterized by θ.
τ	A complete interaction trajectory $(s_1, a_1, \ldots, s_K, a_K)$.
State Features	
q_j	The current task description, updated with information from conversation history.
σ_j	Task uncertainty score at turn j, $\sigma_j \in [0, 1]$, reflecting the incompleteness of the specification.
d_j	The dialogue turn index used as a feature in the state.
r_j	Binary indicator of whether the user rejected the previous turn's action, $r_j \in \{0, 1\}$.
p	The encoded user persona, influencing patience and expertise.
Reward & Cost	
$R(\tau)$	The total trajectory-level reward.
R_{task}	Terminal reward based on the success of the final task execution.
R_t	Per-turn reward combining task success and interruption cost.
$C_{\text{interrupt}}$	Cumulative cost incurred from user interruptions via clarification.
$w_{\text{task}}, w_{\text{interrupt}}$	Weight coefficients for task success and interruption cost respectively.
n_t	Number of clarification questions asked in turn t.
b_t	Binary indicator of whether clarification questions were asked, $b_t \in \{0, 1\}$.
α_t	Binary indicator of whether the user answered a question, $\alpha_t \in \{0, 1\}$.
λ, γ, δ	Cost parameters for the per-turn cost function: λ (base cost), γ (reward for effective clarification), δ (penalty for rejection).
h_{edge}	Binary indicator of whether edge case information was acquired, $h_{\text{edge}} \in \{0, 1\}$.
User Simulation	
p_{base}	The base patience level characteristic of a specific persona.
t	The dialogue turn index used in the patience decay model.
β	The exponential decay rate of user patience across dialogue turns ($\beta = 0.85$).
c_t	Answer clarity at turn t, determined by persona expertise level.

Table 2: Summary of notations used in the TactfulLLM framework.